

The Ultimate Deep-Dive: Modular Architecture & CRUD Anatomy

Author: Abdul Mazid

Profile: Full-Stack MERN Developer | Distributed Systems Enthusiast

Date: May 16, 2026

একজন বিশ্বসেরা ইঞ্জিনিয়ার হতে হলে কোডের শুধু আউটপুট দেখলে হয় না, বরং "প্রতিটি লাইন কী কাজ করছে এবং কেন করছে"—তার পেছনের বিজ্ঞান বুঝতে হয়। এই নোটে আমি আমাদের প্রোজেক্টের পুরো মডুলার আর্কিটেকচার এবং **CRUD (Create, Read, Update, Delete) অপারেশনের প্রতিটি অক্ষর** "মাইক্রোস্কোপিক লেন্ডেলে" ধরে ধরে এক্সপ্লেন করেছি।

১. মডুলার আর্কিটেকচার (MVC Pattern) এর ফ্লো

একটি রিকোয়েস্ট যখন সার্ভারে আসে, তখন তার জার্নিটা ঠিক এমন হয়:

- app.ts (The Traffic Police):** রিকোয়েস্ট রিসিভ করে এবং পথ অনুযায়ী রাউটারে পাঠায় (যেমন `app.use('/api/users', userRoute)`).
- route.ts (The Router):** মেথড (GET/POST) দেখে নির্দিষ্ট কন্ট্রোলারকে কল করে।
- controller.ts (The Manager):** ফ্রন্টএন্ড থেকে ডাটা (`req.body`) রিসিভ করে সার্ভিসের কাছে পাঠায় এবং সার্ভিস থেকে ডাটা পেলে সুন্দর JSON রেসপন্স পাঠায়।
- service.ts (The Worker/Brain):** ডাটাবেসের পুল (`pool.query`) ব্যবহার করে সরাসরি SQL কুয়েরি চালায় এবং লজিক হ্যান্ডেল করে।

২. CRUD অপারেশনের মাইক্রোস্কোপিক ব্যবচ্ছেদ (অক্ষরে অক্ষরে)

এই অংশে আমরা `user.service.ts` ফাইলে থাকা Raw SQL কুয়েরিগুলোকে বেসিক থেকে অ্যাডভান্স লেন্ডেলে এক্সপ্লেন করবো।

● ১. Create (ডাটা তৈরি করা)

কোড:

```
const result = await pool.query(\n  INSERT INTO users(name,email,password,age) VALUES($1,$2,$3,$4) RETURNING *`,\n  [name, email, password, age]\n);
```

অক্ষরে অক্ষরে ব্যাখ্যা:

- await pool.query(...):** Node.js সিঙ্গেল-থ্রেডেড। ডাটাবেসে ডাটা সেভ হতে একটু সময় লাগে। `await` কিওয়ার্ডটি ইভেন্ট লুপকে (Event Loop) বলে দেয়, "যতক্ষণ ডাটাবেস থেকে রিপ্লাই না আসে, তুমি এই ফাংশনকে এখানে পজ (Pause) করে রাখো, কিন্তু অন্য রিকোয়েস্ট ব্লক করো না।"

- **INSERT INTO users(...):** এটি SQL এর কমান্ড। ডাটাবেসকে নির্দেশ দিচ্ছে **users** টেবিলে নতুন সারি (Row) ঢোকাতে। ব্রাকেটের ভেতরে (**name,email,password,age**) দেওয়ার কারণ হলো, ডাটাবেসকে নির্দিষ্ট করে বলা যে আমরা ঠিক কোন কোন কলামে ডাটা দিতে চাচ্ছি (যেমন **id** বা **created_at** আমরা দিচ্ছি না, কারণ ওগুলো ডাটাবেস অটোমেটিক জেনারেট করবে)।
- **VALUES(\$1,\$2,\$3,\$4):** এটি হলো **Parameterized Query**। আমরা সরাসরি ভেরিয়েবল না বসিয়ে **\$1, \$2** বসিয়েছি। এটি SQL Injection হ্যাকিং থেকে বাঁচার একমাত্র উপায়। ডাটাবেস প্রথমে কুয়েরি স্ট্রাকচারটা সেভ করে, এরপর ইনপুটগুলোকে "শুধুমাত্র ডাটা" হিসেবে বসায়।
- **[name, email, password, age]:** এই অ্যারেটি হলো ঐ **\$1, \$2** এর সিরিয়াল ভ্যালু। **\$1** এর জায়গায় **name** বসবে, **\$2** এর জায়গায় **email** বসবে।
- **RETURNING *:** ইনসার্ট করার পর ডাটাবেস সাধারণত কোনো ডাটা ফেরত দেয় না (শুধু **INSERT 0 1** মেসেজ দেয়)। **RETURNING *** লেখার মানে হলো, "যে ডাটাটি মাত্র সেভ হলো, তার আইডি এবং ডেটসহ পুরো ডাটাটি আমাকে এখনই ফেরত দাও।" এর ফলে আমাদের দ্বিতীয়বার **SELECT** কুয়েরি চালাতে হয় না, পারফরম্যান্স বাড়ে।

● ২. Read (ডাটা পড়া বা আনা)

কোড:

```
const result = await pool.query(`SELECT * FROM users WHERE id=$1`, [id]);
```

অক্ষরে অক্ষরে ব্যাখ্যা:

- **SELECT *:** * মানে হলো "All Columns"। এটি ডাটাবেসকে বলছে ওই টেবিলের সবগুলো কলাম (**id, name, email** ইত্যাদি) নিয়ে আসতে। (প্রোডাকশন লেভেলে শুধু প্রয়োজনীয় কলাম আনা ভালো, যেমন **SELECT name, email FROM...**)।
- **FROM users:** কোন টেবিল থেকে আনবে, তা নির্দিষ্ট করা।
- **WHERE id=\$1:** এটি হলো ফিল্টার (Filter) বা কন্ডিশন। ডাটাবেসে যদি ১ মিলিয়ন ইউজার থাকে, **WHERE** ক্লজটি প্রাইমারি-কী (B-Tree Indexing) ব্যবহার করে $O(\log n)$ টাইমে নির্দিষ্ট ইউজারকে খুঁজে বের করে আনে।

● ৩. Update (ডাটা এডিট বা পরিবর্তন করা)

কোড:

```
const result = await pool.query(`UPDATE users SET name=COALESCE($1, name), age=COALESCE($3, age) WHERE id=$5 RETURNING *`, [name, password, age, is_active, id]);
```

অক্ষরে অক্ষরে ব্যাখ্যা:

- **UPDATE users SET:** ডাটাবেসকে বলছে, "users টেবিলে ডাটা আপডেট করবো, চলো সেট (SET) করি।"
- **name=COALESCE(\$1, name):** এটি এই প্রজেক্টের সবচেয়ে মাস্টারপিস লজিক। **COALESCE** (কোয়ালেস) হলো একটি SQL ফাংশন, যা ব্রাকেটের ভেতরের কমা-সেপারেটেড ভ্যালুগুলোর মধ্যে প্রথম যেই ভ্যালুটি **NULL** নয়, সেটিকে পিক (Pick) করে।

- **লজিক**: যদি ফ্রন্টএন্ড থেকে ইউজার **name** আপডেট করার জন্য কোনো ডাটা না পাঠায়, তবে **\$1** এর মান হবে **undefined/NULL**। তখন **COALESCE** দেখবে প্রথমটি **NULL**, তাই সে দ্বিতীয় আর্গুমেন্ট (অর্থাৎ ডাটাবেসের ওই কলামের আগের **name**) কেই পিক করবে।
- **ফলাফল**: ফ্রন্টএন্ড থেকে যে ডাটাগুলো পাঠানো হবে, শুধু সেগুলোই আপডেট হবে। বাকিগুলো আগের মতোই থেকে যাবে। এটি **Partial Update** বা **PATCH** রিকোয়েস্টের জন্য একটি পারফেক্ট, থ্রেড-সেইফ সল্যুশন।
- **WHERE id=\$5**: কার ডাটা আপডেট হবে? যার **id** ৫ নম্বর পজিশনের ভ্যালুর সাথে মিলবে। যদি **WHERE** না দেওয়া হয়, তবে ডাটাবেসের সব ইউজারের নাম আপডেট হয়ে যাবে!

8. Delete (ডাটা মুছে ফেলা)

কোড:

```
const result = await pool.query(`DELETE FROM users WHERE id=$1`, [id]);

// Controller এ চেক করা হয়:
if (result.rowCount === 0) { throw new Error('User Not Found') }
```

অক্ষরে অক্ষরে ব্যাখ্যা:

- **DELETE FROM users**: এটি টেবিল থেকে পুরো রো (Row) মুছে ফেলার কমান্ড।
- **কেন RETURNING * নেই?** ডিলিট হয়ে যাওয়ার পর ডাটাবেসে তো আর ডাটাটির অস্তিত্বই নেই, তাই সে ফেরত পাঠাবে কী? এই কারণে ডিলিটের ক্ষেত্রে আমরা **RETURNING *** ব্যবহার করি না।
- **result.rowCount (কন্ট্রোলারের লজিক)**: ডিলিট কুয়েরি ডাটা রিটার্ন না করলেও, সে কয়টি ডাটা ডিলিট করতে পেরেছে, তার একটা নাম্বার **rowCount** প্রপারটিতে পাঠায়। যদি ডাটাবেসে ওই আইডি'র কেউ না থাকে, তবে ডিলিট হবে ০ টি। তখন **rowCount === 0** হবে, এবং আমরা বুঝতে পারবো ইউজারটি ভুয়া, তাই আমরা ক্লায়েন্টকে **404 Not Found** এরর পাঠিয়ে দিই।

৩. রিলেশনাল ডাটাবেস কনসেপ্ট (Profiles)

ON DELETE CASCADE এর ম্যাজিক

```
user_id INT UNIQUE REFERENCES users(id) ON DELETE CASCADE
```

- **REFERENCES users(id) (Foreign Key)**: এটি গ্যারান্টি দেয় যে, **users** টেবিলে অ্যাকাউন্ট না থাকলে কেউ **profiles** টেবিলে ডাটা সেভ করতে পারবে না।
- **ON DELETE CASCADE**: এটি ডাটাবেসের একটি স্বয়ংক্রিয় ট্রিগার। যখন আমরা **user.service.ts** থেকে **DELETE FROM users** কুয়েরি চালিয়ে কোনো ইউজারকে ডিলিট করি, তখন ডাটাবেস নিজে থেকেই ওই ইউজারের **user_id** ধরে **profiles** টেবিল থেকেও ডাটা মুছে দেয়। নোড.জেএস (Node.js) কে এটার জন্য আলাদা কোনো কোড লিখতে হয় না। এটি ডাটাবেস লেভেলের ১০০% গ্যারান্টিড ক্লিন-আপ প্রসেস।

৪. সিকিউরিটি এবং অথেনটিকেশন (Security & Auth)

মডিউল-৮ এবং ৯-এ আমরা এপিআই-কে সিকিউর করার জন্য বেশ কিছু নতুন প্রযুক্তি যুক্ত করেছি:

8.১ Password Hashing (bcryptjs)

আমরা ডাটাবেসে প্লেইন-টেক্সট (যেমন "12345") পাসওয়ার্ড সেভ করি না। `user.service.ts`-এ আমরা `bcryptjs` ব্যবহার করেছি:

```
const hashPassword = await bcrypt.hash(password, 10);
```

- `bcrypt.hash(..., 10)`: এটি পাসওয়ার্ডটিকে এনক্রিপ্ট করে একটি হিজিবিজি স্ট্রিং (যেমন `$2a$10$...`) বানিয়ে ফেলে। `10` হলো "Salt Round"। এর মানে হলো এটি পাসওয়ার্ডটিকে ১০ বার হ্যাশ করে, যা হ্যাকারদের Brute-Force অ্যাটাক করা অসম্ভব করে তোলে।

8.২ JSON Web Token (JWT)

ইউজার যখন লগইন করে, তখন আমরা `jsonwebtoken` ব্যবহার করে তাকে একটি ডিজিটাল আইডি কার্ড (টোকেন) দিই।

- **কেন JWT?** এটি স্টেটলেস (Stateless)। অর্থাৎ, ইউজার লগইন আছে কি না, তা চেক করার জন্য আমাদের ডাটাবেসে কুয়েরি চালাতে হয় না। টোকেনটি সার্ভারে আসলে সার্ভার তার `secret` কি (Key) দিয়ে টোকেনটি ভেরিফাই করলেই বুঝতে পারে ইউজারটি আসল কি না।

8.৩ Custom Auth Middleware (`src/middleware/auth.ts`)

```
const decoded = jwt.verify(token as string, config.secret as string) as Jwpayload;  
req.user = decoded;  
next();
```

- **কী কাজ করছে?** এটি একটি পাহারাদারের মতো। যখন কোনো রাউটকে আমরা প্রটেক্ট করতে চাই (যেমন `router.get('/', auth(), userController...)`), তখন রিকোয়েস্টটি প্রথমে এই মিডলওয়্যারে আসে।
- সে প্রথমে চেক করে টোকেন আছে কি না, তারপর টোকেন ভেরিফাই করে।
- এরপর সে ডাটাবেসে চেক করে ইউজার এখনো অ্যাক্টিভ আছে কি না (`!user.is_active`)। যদি সব ঠিক থাকে, তবে সে `next()` কল করে রিকোয়েস্টটিকে কন্ট্রোলারের কাছে যেতে দেয়।

8.8 Logger Middleware

```
app.use(logger);
```

এটি একটি গ্লোবাল মিডলওয়্যার। সার্ভারে যত রিকোয়েস্টই আসুক না কেন, এটি রিকোয়েস্টের মেথড (GET/POST) এবং টাইমস্ট্যাম্প ধরে একটি `logger.txt` ফাইলে সেভ করে রাখে। প্রোডাকশন সার্ভারে কোন ইউজার কখন হিট করেছে, তার ট্র্যাকিং রাখার জন্য এটি খুবই জরুরি।

এই হলো একটি এন্টারপ্রাইজ গ্রেড ব্যাকএন্ড প্রজেক্টের প্রতিটি অক্ষরের ইঞ্জিনিয়ারিং ব্যাখ্যা!